MAIN CODE

```
module async_fifo_8x8 (
    input        wr_clk,
    input        rd_clk,
    input        rst,
    input        wr_en,
    input        rd_en,
    input  [7:0]  data_in,
    output reg [7:0] data_out,
    output reg    full,
    output reg    empty
);

    parameter DATA_WIDTH = 8;
    parameter ADDR_WIDTH = 3;
    parameter DEPTH      = 8;

    reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];

    reg [ADDR_WIDTH:0] wr_bin, rd_bin;
    reg [ADDR_WIDTH:0] wr_gray, rd_gray;

    reg [ADDR_WIDTH:0] rd_gray_sync1, rd_gray_sync2;
    reg [ADDR_WIDTH:0] wr_gray_sync1, wr_gray_sync2;

    wire [ADDR_WIDTH:0] wr_bin_next;
    wire [ADDR_WIDTH:0] rd_bin_next;

    wire [ADDR_WIDTH:0] wr_gray_next;
```

```
wire [ADDR_WIDTH:0] rd_gray_next;

wire full_next;
wire empty_next;

// Next binary pointers
assign wr_bin_next =
wr_bin + {{ADDR_WIDTH{1'b0}}, (wr_en && !full)};

assign rd_bin_next =
rd_bin + {{ADDR_WIDTH{1'b0}}, (rd_en && !empty)};

// Binary-to-Gray conversion
assign wr_gray_next = (wr_bin_next >> 1) ^ wr_bin_next;
assign rd_gray_next = (rd_bin_next >> 1) ^ rd_bin_next;

// Predict full after the next write-pointer update
assign full_next =
   (wr_gray_next ==
    {~rd_gray_sync2[ADDR_WIDTH:ADDR_WIDTH-1],
     rd_gray_sync2[ADDR_WIDTH-2:0]});

// Predict empty after the next read-pointer update
assign empty_next = (rd_gray_next == wr_gray_sync2);

// Write clock domain
always @(posedge wr_clk) begin
   if (rst) begin
      wr_bin  <= 0;
```

```
        wr_gray <= 0;
        full    <= 0;
    end
    else begin
        if (wr_en && !full)
            mem[wr_bin[ADDR_WIDTH-1:0]] <= data_in;

        wr_bin  <= wr_bin_next;
        wr_gray <= wr_gray_next;
        full    <= full_next;
    end
end

// Read clock domain
always @(posedge rd_clk) begin
    if (rst) begin
        rd_bin   <= 0;
        rd_gray  <= 0;
        data_out <= 0;
        empty    <= 1;
    end
    else begin
        if (rd_en && !empty)
            data_out <= mem[rd_bin[ADDR_WIDTH-1:0]];

        rd_bin  <= rd_bin_next;
        rd_gray <= rd_gray_next;
        empty   <= empty_next;
    end
```

```
    end

    // Read-pointer synchronizer: read domain -> write domain
    always @(posedge wr_clk) begin
        if (rst) begin
            rd_gray_sync1 <= 0;
            rd_gray_sync2 <= 0;
        end
        else begin
            rd_gray_sync1 <= rd_gray;
            rd_gray_sync2 <= rd_gray_sync1;
        end
    end

    // Write-pointer synchronizer: write domain -> read domain
    always @(posedge rd_clk) begin
        if (rst) begin
            wr_gray_sync1 <= 0;
            wr_gray_sync2 <= 0;
        end
        else begin
            wr_gray_sync1 <= wr_gray;
            wr_gray_sync2 <= wr_gray_sync1;
        end
    end

endmodule
```

TESTBENCH

```
`timescale 1ns/1ps

module tb_async_fifo_8x8;

    reg wr_clk, rd_clk, rst;
    reg wr_en, rd_en;
    reg [7:0] data_in;

    wire [7:0] data_out;
    wire full, empty;

    reg [7:0] model [0:255];

    integer wr_model_ptr;
    integer rd_model_ptr;
    integer errors;
    integer i;

    reg [7:0] expected_data;

    async_fifo_8x8 dut (
        .wr_clk  (wr_clk),
        .rd_clk  (rd_clk),
        .rst     (rst),
        .wr_en   (wr_en),
        .rd_en   (rd_en),
        .data_in (data_in),
        .data_out(data_out),
```

```
    .full    (full),
    .empty   (empty)
);

// Independent clocks
initial begin
    wr_clk = 0;
    forever #5 wr_clk = ~wr_clk;  // 10 ns period
end

initial begin
    rd_clk = 0;
    forever #7 rd_clk = ~rd_clk;  // 14 ns period
end

// Write task
task write_data;
    input [7:0] din;
    begin
        @(negedge wr_clk);

        while (full === 1'b1)
            @(negedge wr_clk);

        if (full !== 1'b0) begin
            $display("ERROR: FULL flag is unknown");
            errors = errors + 1;
        end
```

```
        wr_en   = 1;
        data_in = din;

        // Record the expected data before the write edge
        model[wr_model_ptr] = din;
        wr_model_ptr = wr_model_ptr + 1;

        @(negedge wr_clk);
        wr_en = 0;
    end
endtask

// Read task
task read_data;
    reg [7:0] exp;
    begin
        @(negedge rd_clk);

        while (empty === 1'b1)
            @(negedge rd_clk);

        if (empty !== 1'b0) begin
            $display("ERROR: EMPTY flag is unknown");
            errors = errors + 1;
        end

        exp = model[rd_model_ptr];

        rd_en = 1;
```

```
        @(posedge rd_clk);
        #1;

        if (data_out !== exp) begin
            $display(
                "ERROR: Expected %02h, received %02h at time %0t",
                exp, data_out, $time
            );
            errors = errors + 1;
        end
        else begin
            $display(
                "PASS: Read %02h at time %0t",
                data_out, $time
            );
        end

        rd_model_ptr = rd_model_ptr + 1;

        @(negedge rd_clk);
        rd_en = 0;
    end
endtask

// Check empty flag after read-pointer synchronization
task check_empty;
    begin
        wait (empty === 1'b1);
```

```
        #1;

        if (rd_model_ptr != wr_model_ptr) begin
            $display(
                "ERROR: EMPTY asserted with unread data"
            );
            errors = errors + 1;
        end
        else begin
            $display("PASS: FIFO is empty");
        end
    end
endtask

initial begin
    rst         = 1;
    wr_en       = 0;
    rd_en       = 0;
    data_in     = 0;
    wr_model_ptr = 0;
    rd_model_ptr = 0;
    errors      = 0;

    // Apply reset
    repeat (4) @(negedge wr_clk);
    rst = 0;

    // Allow the read domain to observe the reset
    repeat (3) @(negedge rd_clk);
```

```
if (empty !== 1'b1) begin
   $display("ERROR: FIFO should be empty after reset");
   errors = errors + 1;
end

// Test 1: Fill FIFO with eight words
$display("\nTEST 1: Write eight words");

for (i = 0; i < 8; i = i + 1)
 write_data(8'h10 + i[7:0]);

// Allow registered FULL to update
wait (full === 1'b1);
$display("PASS: FIFO full flag asserted");

// Test 2: Overflow protection
// Hold write enable while full; this write must be rejected
@(negedge wr_clk);
wr_en   = 1;
data_in = 8'hFF;

repeat (2) @(negedge wr_clk);

if (full !== 1'b1) begin
   $display("ERROR: FULL flag deasserted unexpectedly");
   errors = errors + 1;
end
else begin
```

```
    $display("PASS: FIFO rejects writes while full");
end

wr_en = 0;

// Test 3: Read all eight words in order
$display("\nTEST 3: Read eight words");

for (i = 0; i < 8; i = i + 1)
    read_data();

check_empty();

// Test 4: Underflow protection
@(negedge rd_clk);
rd_en = 1;

repeat (2) @(negedge rd_clk);

if (empty !== 1'b1) begin
    $display("ERROR: EMPTY flag incorrect");
    errors = errors + 1;
end
else begin
    $display("PASS: FIFO rejects reads while empty");
end

rd_en = 0;
```

```
        // Test 5: Concurrent writes and reads
        $display("\nTEST 5: Concurrent read/write operations");

        fork
    begin : WRITE_PROCESS
        integer wi;
        reg [7:0] write_value;

        for (wi = 0; wi < 20; wi = wi + 1) begin
            write_value = 8'h80 + wi;
            write_data(write_value);
        end
    end

    begin : READ_PROCESS
        integer ri;

        for (ri = 0; ri < 20; ri = ri + 1)
            read_data();
    end
join

        check_empty();

        // Final result
        if (errors == 0)
            $display("\nTEST PASSED: All checks successful");
        else
            $display("\nTEST FAILED: %0d errors detected", errors);
```

```
        $finish;
    end

endmodule
```